

MPLADS-SATHI

SIH 2026 — Supplementary Technical Deep-Dive

Recommended Tech Stack, ML Training Methodology & AI Integration System for Web & Mobile

Prepared as a focused engineering reference, extending the full MPLADS-SATHI documentation

Covers: production-grade stack selection, per-model training methodology, MLOps lifecycle, and real-time AI serving to React web dashboards and a React Native mobile app.

Document Version 1.0 • August 2026

Table of Contents

1. Scope and Design Goals
2. Recommended Full Tech Stack (Web + Mobile)
3. Mobile Framework Decision: React Native vs Flutter
4. AI/ML Model Training Methodology
5. MLOps: Experiment Tracking, Versioning & Continuous Retraining
6. AI Integration Architecture — Web Application
7. AI Integration Architecture — Mobile Application
8. Offline-First Design for Field Inspectors
9. API Contracts and Sample Integration Code
10. Security, Scaling and Cost Considerations
11. Summary Recommendation Table

1. Scope and Design Goals

This document answers three specific engineering questions raised in continuation of the MPLADS-SATHI proposal: (a) what is the single best, defensible technology stack for both the web dashboards and the mobile app; (b) exactly how should the AI/ML models be trained, validated, and kept accurate over time; and (c) how should AI capability be wired into the web and mobile clients so that risk scores, alerts, and explanations reach the right stakeholder in real time. Every recommendation below is chosen for three properties simultaneously: (1) feasible to demonstrate within a 36-hour hackathon, (2) credible as a production architecture a government CTO would actually approve, and (3) consistent with the modular, explainable, human-in-the-loop design principles established in the main MPLADS-SATHI documentation.

1.1 Non-Negotiable Constraints Driving These Choices

- Explainability is mandatory — every model choice below supports SHAP or equivalent feature-attribution, ruling out unnecessarily opaque deep architectures where simpler models perform comparably.
- Field usage is often low-connectivity — District Authority inspectors work in rural areas, so the mobile stack must support offline-first operation, not just a thin wrapper over the web app.
- Multi-role access at national scale — the same backend must serve MPs, District Authorities, State Nodal Officers, the Ministry, and citizens, each with different data visibility, so the API layer must be RBAC-native from day one.
- Government hosting compatibility — the stack must deploy cleanly on NIC Cloud/MeghRaj in production while remaining runnable on a laptop or free-tier cloud for the hackathon demo.

2. Recommended Full Tech Stack (Web + Mobile)

The table below is the single consolidated stack recommendation — the same one used in the main MPLADS-SATHI documentation, extended here with explicit mobile-layer choices and the reasoning tying each layer together end-to-end.

2.1 Backend & AI Serving

Component	Technology	Why This, Specifically
API Framework	FastAPI (Python 3.11+)	Async-native (critical for concurrent risk-score requests from thousands of MPs/districts), automatic OpenAPI docs that both the web and mobile teams can code against immediately, and native Pydantic validation that catches malformed government data at the boundary rather than deep in a pipeline.
ML Serving	FastAPI inference micro-service + MLflow Model Registry, optionally fronted by BentoML for packaging	Keeps the CRUD API and the ML inference API as separately scalable services — a spike in dashboard traffic never slows down model inference, and vice versa.
Database (transactional + geospatial)	PostgreSQL 16 + PostGIS	One database engine handles both relational work-records and geospatial queries (duplicate-work proximity, constituency-boundary joins) without a second specialised geo-database — reduces

Component	Technology	Why This, Specifically
		operational complexity for a lean hackathon-to-pilot team.
Analytics / Warehouse	PostgreSQL star schema (hackathon/pilot) → ClickHouse (national scale)	Start simple; ClickHouse is a drop-in upgrade only once query volume genuinely requires it — avoids premature infrastructure complexity.
Feature Store	Feast (open-source)	Guarantees the exact feature computation used offline during model training is reused online during real-time scoring — eliminates the single most common cause of ML production bugs (training/serving skew).
Message Queue	RabbitMQ (or managed equivalent)	Decouples slow operations (photo forensic checks, batch re-scoring) from the request/response cycle so dashboards stay fast.
Object Storage	S3-compatible bucket (NIC Cloud object storage in production)	Immutable raw-data snapshots and uploaded inspection photos, both required for audit trail integrity.

2.2 Web Frontend

Component	Technology	Why This, Specifically
Authority Dashboards (MP/DA/State/Ministry)	React 18 + TypeScript + Vite, Tailwind CSS, Recharts + Leaflet.js	TypeScript catches integration errors against the FastAPI OpenAPI schema at build time; Recharts covers standard trend/comparison charts while Leaflet renders the risk-heatmaps and geo-work-maps described in Module C.
Public Citizen Portal	Next.js 14 (App Router, SSR)	Server-side rendering keeps the public transparency pages fast on low-bandwidth mobile browsers in rural constituencies, and is SEO-indexable so citizens can find their constituency's data via search.
State Management	TanStack Query (React Query)	Purpose-built for server-state (risk scores, alerts) with built-in caching/refetching — avoids hand-rolled Redux boilerplate for what is fundamentally an API-driven dashboard.
Component Library	shadcn/ui (Tailwind-based, not a runtime dependency)	Accessible, themeable components that ship as source code the team owns outright — important for a government platform where long-term maintainability matters more than short-term velocity.

2.3 Mobile Frontend

Component	Technology	Why This, Specifically
Mobile Framework	React Native (with Expo for hackathon velocity, bare workflow for production)	Single codebase for Android and iOS; shares TypeScript types and API-client code directly with the React web dashboard, meaningfully cutting development time for a small hackathon team — full reasoning in Section 3.
Offline Storage	WatermelonDB or SQLite (via expo-sqlite)	Enables the offline-first inspection workflow described in Section 8 — a District Authority inspector must be able to view assigned risk-ranked works and submit inspection findings with zero network connectivity.
Push Notifications	Firebase Cloud Messaging (FCM)	Free, reliable, cross-platform push delivery for high-risk alerts — the standard choice for GovTech mobile apps in India.
On-Device Inference	TensorFlow Lite (optional, Phase 2+)	Allows a lightweight photo-quality/duplicate pre-check to run directly on the inspector's phone before upload,

Component	Technology	Why This, Specifically
		saving bandwidth in low-connectivity districts — detailed in Section 8.2.
Maps on Mobile	react-native-maps (Google Maps / Bhuvan tile layer)	Consistent mapping experience with the web Leaflet views, using the same underlying geo-coordinates.

3. Mobile Framework Decision: React Native vs Flutter

Both React Native and Flutter are credible, production-grade choices for a GovTech field-inspection app. The recommendation below is React Native, with the reasoning made explicit rather than asserted, since a jury may reasonably ask why Flutter was not chosen.

Factor	React Native	Flutter
Code-sharing with web dashboard	Shares TypeScript types, API client, validation logic, and business rules directly with the React/Next.js web codebase	No code sharing — Dart is a separate language from the TypeScript web stack, effectively doubling logic-maintenance surface
Team learning curve for a hackathon	Low, if the team already knows React for the web dashboard (very likely for an SIH team)	Requires learning Dart from scratch under hackathon time pressure
Offline-first data layer maturity	Mature ecosystem (WatermelonDB, Realm, SQLite bindings) purpose-built for offline-sync apps	Also mature (Drift, Isar) but with a smaller pool of GovTech reference implementations in India
Performance for this use case	More than sufficient — this app is data/forms/maps-heavy, not a graphics-intensive game where Flutter's rendering edge matters	Marginally better raw rendering performance, not a decisive factor here
Ecosystem precedent in Indian GovTech	Widely used (e.g., several UMANG-ecosystem and state-government apps)	Growing but less dominant in Indian public-sector mobile deployments to date

Conclusion: React Native is recommended primarily because it lets the same small team that builds the React web dashboard extend directly into mobile without a second technology stack, which is decisive for both a 36-hour hackathon team and a lean pilot-phase engineering team. Flutter remains a reasonable alternative and should be named as such if a jury asks — this is a defensible engineering trade-off, not a dismissal of Flutter's merits.

4. AI/ML Model Training Methodology

MPLADS-SATHI is not one model — it is a portfolio of purpose-built detectors, each trained differently because each solves a structurally different problem (text classification, statistical outlier detection, time-to-event prediction, graph analysis, image forensics). This section specifies the training methodology for each, plus the data-splitting and evaluation discipline that applies across all of them.

4.1 Cross-Cutting Training Principles

- Time-based splitting, not random splitting: because MPLADS data is inherently temporal (works are sanctioned and completed over fiscal years), train/validation/test splits must be chronological — train on FY2022–24 data, validate on FY2024–25, test on FY2025–26 — to honestly simulate how the model will perform on genuinely future, unseen works. Random splitting would leak future information into training and produce misleadingly optimistic offline metrics.
- Stratification for rare events: confirmed fraud/violation cases are rare relative to the total work volume (consistent with the CAG-documented ~₹161 crore undocumented-expenditure figure against an ~₹8,318 crore total corpus). Stratified sampling and class-weighting (or techniques like SMOTE for the classical models) are used so rare-but-critical cases are not drowned out during training.

- Label sourcing strategy: true labels come from three sources, ranked by reliability — (1) confirmed CAG audit paragraphs (highest confidence, used as "gold" test labels), (2) District Authority physical inspection outcomes fed back through the platform (Section 5), and (3) weak/heuristic labels from the rule-based detectors in Module A, used only for pre-training / cold-start bootstrapping, never as final ground truth.
- Every model ships with a SHAP explainer: no detector is deployed without a corresponding feature-attribution method, per the explainability-by-design principle established in the main MPLADS-SATHI proposal.

4.2 Model-by-Model Training Specification

4.2.1 NLP Prohibited-Category Classifier

Attribute	Specification
Problem type	Binary/multi-label text classification (prohibited-category present / absent, per category type)
Baseline model	Bilingual keyword/phrase rule engine (already validated at 55% flag rate on real Kandhamal data — see main documentation, Section 7.5)
Upgrade model	Fine-tuned IndicBERT or MuRIL (multilingual, India-focused transformer) on work_description + work_description_hi
Training data	CAG audit paragraphs describing confirmed prohibited-category works (positive examples) + a random sample of confirmed permitted works (negative examples), augmented with the rule-engine's high-confidence flags
Class imbalance handling	Weighted cross-entropy loss; positive class up-weighted given the rarity of confirmed prohibited-category cases in a random sample
Evaluation metric	F1-score on the prohibited-category class (precision and recall equally important: false positives damage MP reputations, false negatives let violations through)
Explainability	Attention-weight / SHAP-for-text (e.g., via the `shap.Explainer` text masker) highlighting which words/phrases drove the flag

4.2.2 Cost-Estimate Anomaly Detector

Attribute	Specification
Problem type	Unsupervised anomaly detection (no reliable "confirmed inflated estimate" label exists at scale)
Model	Isolation Forest on standardised per-unit cost features (cost per sq. m for construction, cost per km for roads, cost per unit for fixed-item works like solar lights)
Training data	Historical cost records grouped by (work_type, district/state), with Schedule-of-Rates (SOR) baseline joined in as a reference feature once Tier-2 data is available
Contamination parameter	Set conservatively (e.g., 2–5%) based on CAG's historical audit-flag rate, and tuned during the Phase 2 pilot against real inspection outcomes rather than guessed
Evaluation approach	Since ground truth is scarce, evaluated primarily via analyst review of top-N flagged outliers during the pilot (precision-at-K), supplemented by any CAG-confirmed cases available as spot-check test labels
Explainability	SHAP values on the Isolation Forest via `TreeExplainer`, showing which cost-feature (e.g., "42% above district per-sq-m median") drove the anomaly score

4.2.3 Delay Prediction Model

Attribute	Specification
Problem type	Binary classification (will this work exceed the 45-day sanction window / overall completion deadline) or survival/time-to-event regression for a finer-grained estimate
Model	Gradient-boosted trees (XGBoost or LightGBM) — chosen over deep learning because the feature set is tabular/structured, dataset size at pilot stage will be modest (thousands, not millions, of records), and tree-based SHAP explanations are fast and exact
Features	District administrative capacity (historical average sanction time), work type, season/month of recommendation, implementing-agency track record, fund-release lag, MP's historical recommendation volume
Training data	Historical work records with actual sanction_date and work_order_date deltas, once Tier-1 data (Section 8 of the main documentation) is integrated via the district pilot
Validation	Time-based k-fold (e.g., expanding-window cross-validation across fiscal quarters) to simulate real deployment conditions
Evaluation metrics	ROC-AUC and precision/recall at the operating threshold used for dashboard alerting, plus calibration curves — a predicted "80% probability of delay" should genuinely correspond to an 80% empirical delay rate
Explainability	SHAP `TreeExplainer` — fast, exact, and specifically well-suited to gradient-boosted tree models

4.2.4 Contractor Network / Fraud-Ring Analysis

Attribute	Specification
Problem type	Graph analytics (not a trained predictive model in the classical sense) plus optional supervised risk-scoring on graph-derived features
Approach	Build a heterogeneous graph (Agency—Contractor—Bank Account—Work) in NetworkX; compute degree/betweenness centrality and connected-component analysis to surface contractors with disproportionate work concentration or shared identity signals (same bank account/PAN under different registered names)
Identity resolution	Fuzzy string matching (e.g., Levenshtein/Jaro-Winkler) on contractor names combined with exact-match on PAN/GST/bank account, to catch blacklisted contractors reappearing under slightly altered names
Optional supervised layer	Once sufficient labelled fraud-ring cases exist (from CAG or pilot inspection outcomes), a gradient-boosted classifier can be trained on graph-derived features (centrality, cluster size) to produce a numeric contractor risk score
Evaluation	Precision-at-K against confirmed blacklist entries and any CAG-documented fraud-ring cases used as a held-out test set

4.2.5 Ghost-Work / Photo-Forensic Verification

Attribute	Specification
Problem type	Image similarity/duplicate detection (unsupervised) + geo-metadata validation (rule-based)
Model	Perceptual hashing (pHash) for near-duplicate/reused photo detection across unrelated works, combined with a pretrained CNN embedding model (e.g., a lightweight ResNet or MobileNet feature extractor) for cosine-similarity search against a photo database
Geo validation	EXIF GPS extraction + reverse geocoding, cross-checked against the claimed work location within a configurable tolerance radius (rule-based, no training required)
Training/fine-tuning data	Not required for the base similarity/EXIF checks; a future fine-tuned classifier for AI-generated-image detection would require a labelled dataset of confirmed synthetic vs. genuine site photos, sourced during the pilot
On-device variant	A quantised MobileNet model (TensorFlow Lite) can run directly on the inspector's phone for an instant "this looks like a possible duplicate" pre-check before upload — see Section 8.2

4.3 Composite Risk Score Training

The final 0–100 composite risk score is not itself a black-box trained model in the initial phase — it is a documented, versioned weighted sum of the individual detector outputs, with weights initially set by domain-expert judgement (informed by CAG audit-severity classifications) and then calibrated during the Phase 2 pilot by comparing predicted risk rankings against actual physical-inspection outcomes (isotonic regression calibration). Only once sufficient labelled outcome data accumulates (Phase 3) does a learned meta-model (e.g., logistic regression or a shallow gradient-boosted model over the detector outputs as features) become appropriate — deliberately avoiding a premature black-box composite score before there is enough real-world feedback to trust it.

5. MLOps: Experiment Tracking, Versioning & Continuous Retraining

A public-fund fraud-detection system cannot be a one-time trained model — it must improve continuously as real inspection outcomes accumulate, while remaining fully auditable (every historical risk score must be traceable to the exact model version that produced it). The following MLOps lifecycle achieves both.

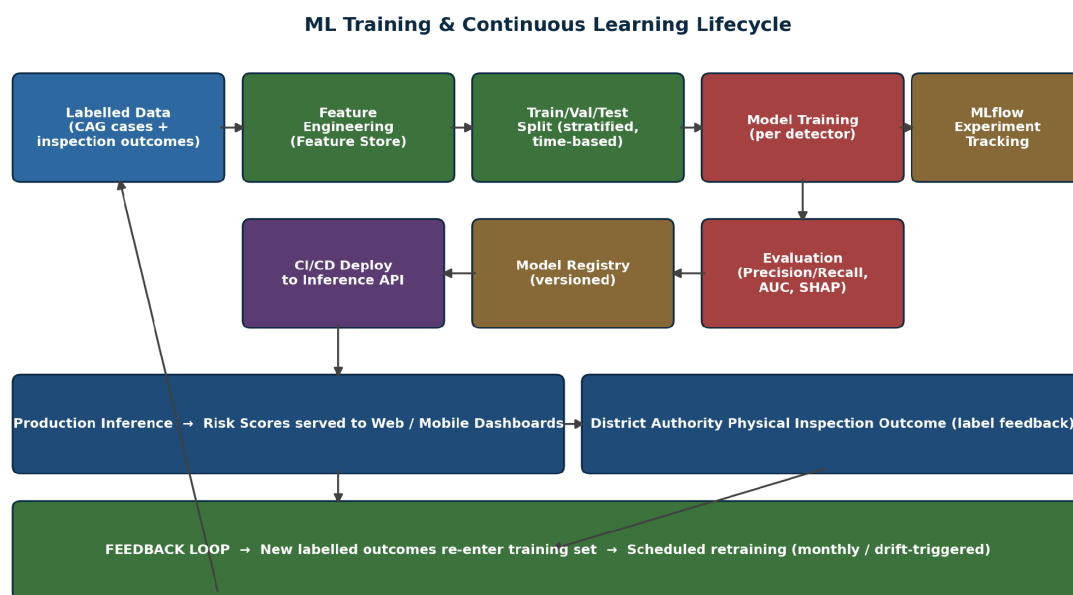


Figure A: The training-to-production-to-feedback lifecycle. District Authority inspection outcomes flow back into the labelled training set, driving scheduled or drift-triggered retraining.

5.1 Experiment Tracking

- MLflow Tracking Server logs every training run's hyperparameters, metrics (F1, AUC, precision-at-K), and artefacts, giving the team a complete, comparable history of every model iteration.
- Each detector (Section 4.2) is tracked as a separate MLflow experiment, since they have different metrics and evaluation cadences.

5.2 Model Registry and Versioning

- Every trained model that passes evaluation thresholds is registered in the MLflow Model Registry with a semantic version and a lifecycle stage (Staging → Production → Archived).
- The Inference Service (Section 2.1) always loads the model marked "Production" for its detector — promoting a new model version is a deliberate, logged action, never a silent redeploy.
- Every risk score persisted to the database stores the exact model version(s) used to compute it, so any flagged work can be traced back to a specific, reproducible model snapshot for audit or appeal — directly supporting the governance requirement established in the main documentation (Section 12.7).

5.3 CI/CD for Models

- A GitHub Actions pipeline runs automated evaluation against a held-out test set on every proposed model update; a model can only be promoted to Production if it meets or exceeds the currently deployed model's metrics on the same test set — preventing silent quality regressions.
- Deployment to the Inference Service is a blue/green rollout: the new model version serves a small percentage of traffic first (canary), with automated rollback if error rates or prediction-distribution drift exceed thresholds.

5.4 Continuous Retraining Triggers

Trigger	Action
Scheduled (monthly)	Standard retraining cycle incorporating the latest month's inspection-outcome feedback into the labelled training set
Data drift detected	Automated statistical drift monitoring (e.g., population stability index on key features) triggers an out-of-cycle retraining review if the incoming work-record distribution shifts materially (e.g., a new state's data onboarded)
Concept drift / accuracy decay	If live precision-at-K (measured against arriving inspection outcomes) drops below a configured threshold, an alert triggers manual review and likely retraining
New Tier-2/Tier-3 data source onboarded	Any newly integrated data source (e.g., Census SC/ST data, PFMS payment history) triggers a full feature-engineering and retraining cycle to incorporate the new signal

5.5 Human-in-the-Loop Governance

No retrained model is auto-promoted to production without a documented sign-off step in the deployment pipeline. This mirrors the main documentation's governance principle (Section 12.7): automation accelerates detection, but a human remains accountable for every consequential model change, exactly as a human remains accountable for every consequential risk-score-triggered inspection.

6. AI Integration Architecture — Web Application

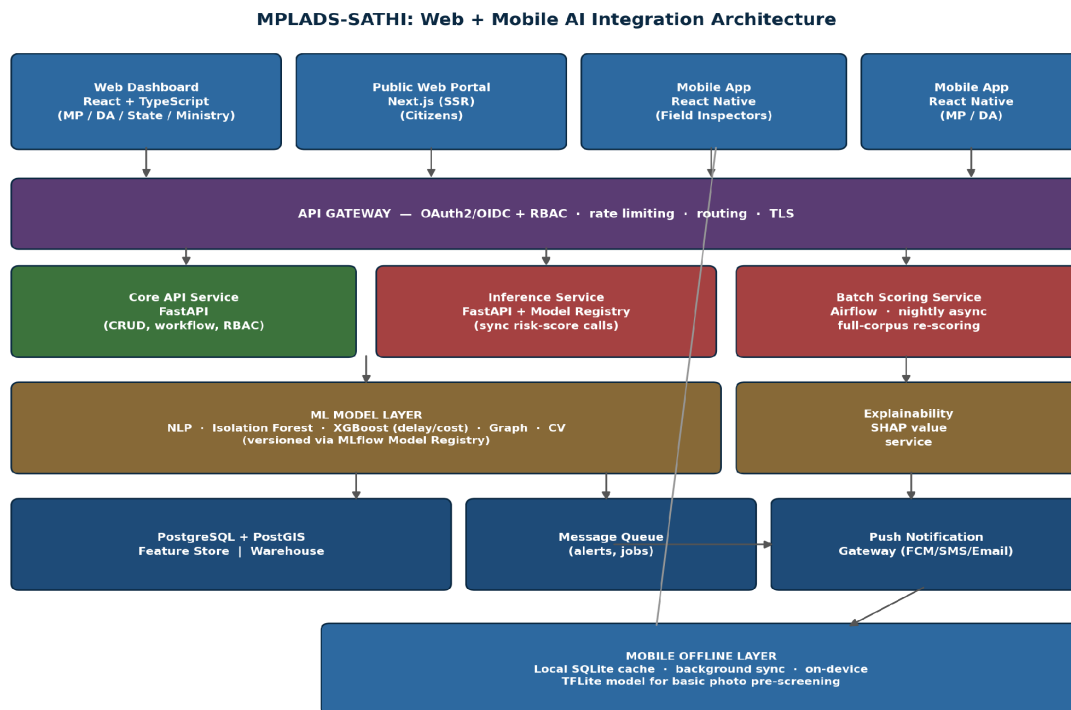


Figure B: End-to-end integration — web dashboards and mobile apps both authenticate through a single API gateway, which routes to the Core API Service (CRUD/workflow) and the Inference Service (AI risk scoring), backed by a versioned ML model layer and SHAP explainability service.

6.1 Two-Service Split: Core API vs Inference API

The Core API Service (work records, workflow, user/role management) and the Inference Service (risk scoring, explanations) are deliberately deployed as separate FastAPI services behind a shared API gateway. This means a burst of MPs checking their dashboards never slows down risk-score computation, and the ML team can deploy new model versions independently of the web team's release cycle.

6.2 Synchronous vs Asynchronous Scoring

- Synchronous: when a District Authority officer opens a single work's detail page, the web app calls the Inference Service directly and receives the risk score + SHAP explanation within the request (target: under 300ms, achievable since most detectors — NLP keyword, cost-cluster, geo-check — are lightweight).
- Asynchronous/batch: nightly, an Airflow job re-scores the full national work corpus (all newly added or updated records) and writes results to PostgreSQL, so that dashboard list/heatmap views (which may show thousands of works at once) read pre-computed scores instead of triggering thousands of live inference calls.

6.3 Sample Integration — React Web Dashboard

The web dashboard consumes the Core API for work data and joins it with pre-computed risk scores, using TanStack Query for caching:

```
// districtDashboard.ts – fetching risk-ranked inspection queue
import { useQuery } from '@tanstack/react-query';

async function fetchInspectionQueue(districtId: string) {
  const res = await fetch(
    `/api/v1/districts/${districtId}/works?sort=risk_score_desc&limit=50`,
    { headers: { Authorization: `Bearer ${getAuthToken()}` } }
  );
  if (!res.ok) throw new Error('Failed to load inspection queue');
  return res.json(); // [{ work_id, risk_score, top_reasons: [...], ... }]
}

export function useInspectionQueue(districtId: string) {
  return useQuery({
    queryKey: ['inspection-queue', districtId],
    queryFn: () => fetchInspectionQueue(districtId),
    staleTime: 5 * 60 * 1000, // 5 min – matches nightly batch-score cadence
  });
}
```

6.4 Real-Time Alerts via WebSocket

For time-sensitive events — a work crossing the 45-day sanction deadline, or a newly uploaded work scoring above the high-risk threshold — the platform pushes updates over a WebSocket channel rather than requiring the dashboard to poll:

```
// alertsSocket.ts
const socket = new WebSocket(`wss://api.mplads-sathi.gov.in/ws/alerts?token=${authToken}`);

socket.onmessage = (event) => {
  const alert = JSON.parse(event.data);
  // alert = { type: 'HIGH_RISK_WORK', work_id, risk_score, district_id }
  showToastNotification(alert);
  queryClient.invalidateQueries(['inspection-queue', alert.district_id]);
};
```

7. AI Integration Architecture — Mobile Application

The mobile app is not a scaled-down version of the web dashboard — it is purpose-built for two distinct user journeys: (1) a District Authority field inspector executing the AI-ranked inspection queue on-site, often with poor connectivity, and (2) an MP or officer checking constituency status on the move.

7.1 Shared API, Role-Aware Client

The mobile app authenticates against the same API gateway and Core/Inference services as the web dashboard (Figure B) — there is no separate mobile-only backend to maintain. The same TypeScript API-client types generated from the FastAPI OpenAPI schema are reused between the React web app and the React Native app, which is one of the concrete productivity benefits of the React Native choice made in Section 3.

7.2 Sample Integration — React Native Inspection Screen

```
// InspectionQueueScreen.tsx
import { useInspectionQueue } from '../hooks/useInspectionQueue';
import { useNetInfo } from '@react-native-community/netinfo';
import { get_cached_queue, cache_queue } from '../offline/db';

export function InspectionQueueScreen({ districtId }) {
  const { isConnected } = useNetInfo();
  const { data, isLoading } = useInspectionQueue(districtId, {
    enabled: isConnected,
  });

  useEffect(() => {
    if (data) cache_queue(districtId, data); // persist for offline use
  }, [data]);

  const queue = data ?? get_cached_queue(districtId); // fall back offline

  return <RiskRankedList items={queue} />;
}
```

7.3 Push Notifications for High-Risk Alerts

Firebase Cloud Messaging delivers server-triggered push notifications to District Authority and MP mobile clients when a work crosses the high-risk threshold or a compliance deadline (45-day sanction window, SC/ST earmarking shortfall) is imminent — surfacing the Module D automated compliance checks described in the main documentation directly on the relevant official's phone rather than requiring them to log into a dashboard.

8. Offline-First Design for Field Inspectors

Given that MPLADS Guidelines mandate 10% physical inspection of works annually, and many such works are in rural or low-connectivity areas, offline capability is not a nice-to-have — it is core to the mobile app actually being used.

8.1 Local-First Data Architecture

- On app open (with connectivity), the current risk-ranked inspection queue for the officer's assigned district is downloaded and persisted to a local WatermelonDB/SQLite store.

- Inspectors can browse work details, view risk explanations, and fill out inspection forms entirely offline; completed inspection records are queued locally with a pending-sync flag.
- A background sync service (using React Native's background fetch APIs) uploads pending inspection records whenever connectivity is restored, with conflict resolution favouring the most recent server state for shared fields and always preserving the inspector's submitted findings.

8.2 On-Device Pre-Screening (Photo Forensics)

Rather than requiring a large photo upload over a poor connection before any feedback is given, a quantised TensorFlow Lite model bundled with the app performs an instant on-device check — flagging obvious duplicate-looking images or missing GPS metadata immediately, before the inspector leaves the site. The full server-side forensic check (Section 4.2.5) still runs once the photo syncs, but this on-device pass avoids wasted return trips caused by unusable photo submissions.

8.3 Sync Conflict Handling

```
// syncEngine.ts (simplified)
async function syncPendingInspections() {
  const pending = await db.get('inspections').query(Q.where('synced', false));
  for (const record of pending) {
    try {
      await api.post(`/api/v1/works/${record.workId}/inspections`, record);
      await record.update((r) => { r.synced = true; });
    } catch (err) {
      // leave unsynced; retried on next connectivity event
      logSyncFailure(record.id, err);
    }
  }
}
```

9. API Contracts and Sample Integration Code

9.1 Core Risk-Score Endpoint Contract

```
GET /api/v1/works/{work_id}/risk
Authorization: Bearer <token>

200 OK
{
  "work_id": "179092",
  "risk_score": 78,
  "confidence_tier": "verification_required",
  "flags": [
    {"type": "religious_keyword", "confidence": "high", "matched": "temple"},
    {"type": "geo_mismatch", "confidence": "medium", "detail": "district=Nayagarh vs constituency=Kandhamal"}
  ],
  "shap_explanation": [
    {"feature": "keyword_match_score", "contribution": 0.34},
    {"feature": "geo_distance_to_constituency_centroid", "contribution": 0.22}
  ],
  "model_version": "nlp-prohibited-v1.3.0"
}
```

9.2 FastAPI Inference Endpoint (Server Side)

```
# inference_service/main.py
from fastapi import FastAPI, Depends
from .model_registry import load_production_model
from .explain import shap_explain

app = FastAPI()
nlp_model = load_production_model('nlp-prohibited')
cost_model = load_production_model('cost-anomaly')

@app.get('/api/v1/works/{work_id}/risk')
async def get_risk(work_id: str, user=Depends(get_current_user)):
    record = await fetch_work_record(work_id)
    check_rbac(user, record) # role-based visibility check
    flags = run_all_detectors(record, nlp_model, cost_model)
    score = composite_risk_score(flags)
    explanation = shap_explain(record, flags)
    return {
        'work_id': work_id, 'risk_score': score,
        'flags': flags, 'shap_explanation': explanation,
        'model_version': nlp_model.version,
    }
```

10. Security, Scaling and Cost Considerations

10.1 Security

- OAuth2/OIDC authentication with RBAC enforced at the API gateway and re-verified at each service — an MP's token can never retrieve another MP's flagged data, and mobile tokens are scoped to the officer's assigned district.
- All traffic over TLS 1.3; data at rest encrypted (AES-256) in PostgreSQL and object storage, consistent with Government of India cloud security baselines.
- Mobile app stores only a short-lived refresh token in secure device storage (Keychain/Keystore), never long-lived credentials, to limit exposure if a device is lost.
- Every inference request and every model-version promotion is written to an immutable audit log, supporting the traceability requirement established in Section 5.2.

10.2 Scaling

- Core API and Inference API are independently horizontally scalable behind the gateway (Kubernetes Horizontal Pod Autoscaler in production), so a national rollout can scale the compute-heavy Inference Service without over-provisioning the lighter Core API.
- Nightly batch scoring (Section 6.2) is designed to complete well within an overnight window even at full national scale (~54,000 estimated works), since the lightweight detectors (NLP, cost-cluster, geo-check) are computationally cheap per record.
- Read-heavy dashboard queries are served from the pre-aggregated analytics warehouse (Section 2.1), keeping the transactional PostgreSQL database free for write-heavy inspection-sync traffic from the mobile app.

10.3 Cost Profile (Directional)

Phase	Infrastructure Cost Driver	Approach
Hackathon MVP	Free-tier cloud (AWS/GCP) + local development	Zero to minimal cost; entirely demonstrable on a laptop plus a free-tier deployment for the live demo
Pilot (1 district)	Small managed PostgreSQL instance, single Kubernetes namespace or simple VM, FCM free tier	Low, predictable monthly cost — modest enough to be self-funded or covered by a small grant during validation
National scale	NIC Cloud/MeghRaj hosting, horizontally scaled services, managed object storage	Government-hosted, consistent with existing budget lines for eSAKSHI/PFMS-class systems; incremental cost justified directly by the inspection-efficiency and fraud-prevention gains in Section 14 of the main documentation

11. Summary Recommendation Table

Layer	Recommended Choice	Key Reason
Backend API	FastAPI (Python)	Async, auto-documented, Pydantic-validated
ML Serving	FastAPI + MLflow Model Registry	Versioned, auditable inference
Database	PostgreSQL + PostGIS	One engine for relational + geospatial
Feature Store	Feast	Eliminates training/serving skew
Web Dashboards	React + TypeScript + Tailwind	Type-safe, fast to build, shared types with mobile
Public Portal	Next.js (SSR)	Fast, SEO-friendly, low-bandwidth-tolerant
Mobile App	React Native (Expo → bare)	Shares code/types with web team; strong offline ecosystem
Offline Store (Mobile)	WatermelonDB / SQLite	Purpose-built for offline-sync apps
Push Notifications	Firebase Cloud Messaging	Free, reliable, cross-platform
On-Device Inference	TensorFlow Lite	Instant photo pre-screening without connectivity
Explainability	SHAP across all models	Consistent, auditable risk-score reasoning
Experiment Tracking	MLflow	Reproducible, comparable training runs
Orchestration	Apache Airflow	Scheduled ingestion, batch scoring, retraining triggers
Deployment	Docker → Kubernetes on NIC Cloud/MeghRaj	Portable from hackathon laptop to national production

This stack, training methodology, and integration design are intentionally consistent with — and a direct technical elaboration of — the architecture proposed in the main MPLADS-SATHI documentation (Sections 9–12), ensuring both documents can be presented together as a single coherent engineering submission.